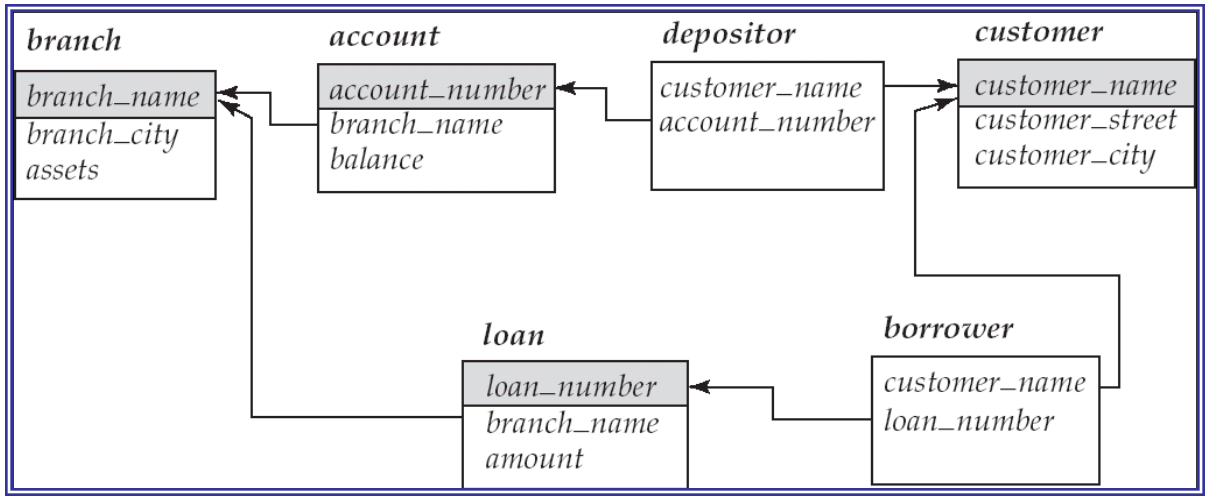
Database Development and Design (SOE201)

Lab 4: Tutorial on Query Evaluation

Dr. Wei Wang

AIAT

Q1



- Given the following information:
  - Number of records of *customer*: 1,000,000
  - Number of blocks of *customer*: 8,000
  - Number of records of *depositor*: 500,000
  - Number of blocks of *depositor: 1,000*
- Assume we use indexed nested loop join for evaluating the join; and use the depositor as outer relation. We also assume that customer have a primary B+-tree index on the join attribute customer-name, with n=100.
  - What is the number of block transfers and seeks, respectively?
  - Assume t_T=0.1ms and t_S=4ms, what is the total cost in terms of time?
  - Together with the nested loop join and block nested loop join, which method is the best in terms of total cost?

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Q2

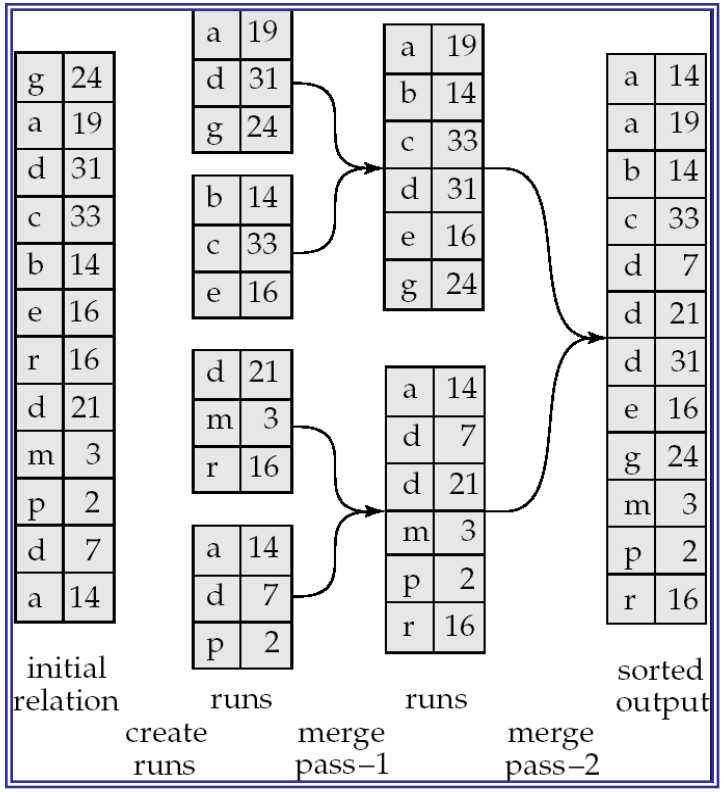
- Consider the following two relations and their catalogue information.
  - i) *customer(customer_Number, name, phone, address)*
  - ii) *transaction(transaction_ID, date, amount, customer_Number)*
- The "*customer_Number*" is the key for the *customer* relation, and "*transaction_ID*" is the key for the *transaction* relation. The *transaction.customer_Number* is the foreign key referencing the *customer* relation. The *customer* relation has 5,000 tuples stored in 500 blocks, and the *transaction* relation contains 80,000 tuples stored in 7,000 blocks. Assume that the relation *transaction* has been sequentially sorted by the key attribute. Answer the following questions.

Q2 cont'd

- Assume that none of the relations can fit in memory, time for one block transfer t_T=0.1ms, and time for one seek t_S=4ms.
- Suppose that a sparse primary B+ tree index (*N*=7) has been created for the *transaction* relation on the *customer_Number* attribute.
- Which of the two algorithms, the block nested loop join and indexed nested loop join, is more efficient to evaluate "*customer* ⋈ *transaction*"? Justify your answer.

Q3

This is about the external sort-merge. Suppose that Memory hold at most three page frames, only one tuple fits in block. The relation needs 12 blocks to store, m= 3, *assume* b_b =1.



Total block transfer = ?

Total seeks = ?

Q4

- Given the following information. Assume relations are not sorted (need to be sorted using external merge sort).
  - Number of records of customer: 1,000,000
  - Number of blocks of customer: 8,000
  - Number of records of depositor: 500,000
  - Number of blocks of depositor: 1,000
- If we use merge join, assume memory size M=100 and the number of blocks allocated in memory for buffering each relation is $b_b=5$, what is the total cost in terms of block transfer and seek?

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Q5

- Given the information from Q4.
- *depositor* is to be used as build input. Partition it into 50 partitions, each of size 20 blocks. This partitioning can be done in one pass. Similarly, partition *customer* into 50 partitions, each of size 160. This is also done in one pass.
- Assuming 20 blocks are allocated for the input buffer and each output buffer (b_b=20)
- What is the total cost (the cost of writing partially filled blocks needs to be considered)?

Q6

- Consider the following two unary relations *r*, b_r=5 and *s*, b_s=3.
- Assume that each block can hold two tuples from r and s; the memory size is 3, and one block is used to buffer the read/write, i.e., b_b=1. Note that if a relation can fit into memory, the external merge sort is not needed.
- Answer the following questions on external merge sort and merge join.
  - (a) How many sorted runs will be produced for relations r and s, respectively?
  - (b) How many merge passes are needed to sort r and s, respectively?
  - (c) Assume that the cost for writing the final results to disk needs to be considered in order to perform the subsequent merge join, what are the numbers of block transfers for sorting r and s, respectively?
  - (d) What is the number of block transfers for the merge join of r and s?
  - (e) What is the total cost in terms of block transfers for the whole process?